

```
import java.rmi.*;
```

```
public interface Search extends Remote {
```

```
    public String query(String search) throws RemoteException;
```

```
}
```

```
import java.rmi.*;
```

```
import java.rmi.server.*;
```

```
public class SearchQuery extends UnicastRemoteObject implements Search {
```

```
    public SearchQuery() throws RemoteException {
```

```
        super();
```

```
    }
```

```
    public String query(String search) throws RemoteException {
```

```
        String result;
```

```
        if (search.equals("Reflection in Java"))
```

```
            result = "Found";
```

```
        else
```

```
            result = "Not Found";
```

```
        return result;
```

```
    }
```

```
}
```

```
import java.net.*;
```

```
import java.io.*;

public class Server {

    private Socket socket = null;
    private ServerSocket server = null;
    private DataInputStream in = null;

    public Server(int port) {

        try {

            server = new ServerSocket(port);

            System.out.println("Server started");
            System.out.println("Waiting for a client ...");

            socket = server.accept();

            System.out.println("Client accepted");

            in = new DataInputStream(
                new BufferedInputStream(socket.getInputStream()));

            String line = "";

            while (!line.equals("Over")) {

                try {

                    line = in.readUTF();
```

```
        System.out.println(line);
```

```
    } catch (IOException i) {
```

```
        System.out.println(i);
```

```
    }
```

```
}
```

```
System.out.println("Closing connection");
```

```
socket.close();
```

```
in.close();
```

```
    } catch (IOException i) {
```

```
        System.out.println(i);
```

```
    }
```

```
}
```

```
public static void main(String args[]) {
```

```
    Server server = new Server(5000);
```

```
}
```

```
}
```

```
import java.io.*;
```

```
import java.net.*;
```

```
public class Client {
```

```
    private Socket socket = null;
```

```
private BufferedReader d = null;
private InputStream input = null;
private DataOutputStream out = null;

public Client(String address, int port) {

    try {

        socket = new Socket(address, port);

        System.out.println("Connected");
        System.out.println("Done with 1st program of DS");

        d = new BufferedReader(new InputStreamReader(System.in));

        out = new DataOutputStream(
            socket.getOutputStream());

    } catch (UnknownHostException u) {

        System.out.println(u);
        return;

    } catch (IOException i) {

        System.out.println(i);
        return;
    }

    String line = "";
```

```

while (!line.equals("Over")) {

    try {

        line = d.readLine();
        out.writeUTF(line);

    } catch (IOException i) {

        System.out.println(i);
    }
}

try {

    input.close();
    out.close();
    socket.close();

} catch (IOException i) {

    System.out.println(i);
}

}

public static void main(String args[]) {

    Client client = new Client("127.0.0.1", 5000);
}
}

```

Practical 3

hello.c

```
#include <stdio.h>
```

```
#include "mpi.h"
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int rank, size;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    printf("Hello, world, I am %d of %d\n", rank, size);
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}
```

send_recv.c

```
#include <stdio.h>
```

```
#include "mpi.h"
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int rank, size;
```

```
    int num = 10;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```

if(rank == 0)
{
    printf("Sending message containing: %d from rank %d\n", num, rank);
    MPI_Send(&num, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);
}
else
{
    printf("at rank %d\n", rank);
    MPI_Recv(&num, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Received message containing: %d at rank %d\n", num, rank);
}

MPI_Finalize();
return 0;
}

```

array_sum_mpi.c

```
#include <stdio.h>
```

```
#include "mpi.h"
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int rank, size;
```

```
    int num[20];
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    for(int i = 0; i < 20; i++)
```

```

    num[i] = i + 1;

if(rank == 0)
{
    int s[4];
    int sum = 0, local_sum = 0;

    printf("Distribution at rank %d\n", rank);

    for(int i = 1; i < 4; i++)
        MPI_Send(&num[i * 5], 5, MPI_INT, i, 1, MPI_COMM_WORLD);

    for(int i = 0; i < 5; i++)
        local_sum += num[i];

    for(int i = 1; i < 4; i++)
        MPI_Recv(&s[i], 1, MPI_INT, i, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    printf("local sum at rank %d is %d\n", rank, local_sum);

    sum = local_sum;

    for(int i = 1; i < 4; i++)
        sum += s[i];

    printf("final sum = %d\n", sum);
}
else
{
    int k[5];
    int local_sum = 0;

```



```
MPI_Recv(k, 5, MPI_INT, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

for(int i = 0; i < 5; i++)
    local_sum += k[i];

printf("local sum at rank %d is %d\n", rank, local_sum);

MPI_Send(&local_sum, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}
```

Prcatical 4

```
import java.util.ArrayList;
```

```
public class BerkeleyClockSync {
```

```
    public static void main(String[] args) {
```

```
        int[] systemClocks = {10, 12, 13, 11, 14};
```

```
        int masterClock = 0;
```

```
        System.out.print("System clocks: ");
```

```
        for (int clock : systemClocks) {
```

```
            System.out.print(clock + " ");
```

```
        }
```

```
        System.out.println();
```

```
        int sum = 0;
```

```
        for (int clock : systemClocks) {
```

```
            sum += clock;
```

```
        }
```

```
        int averageClock = sum / systemClocks.length;
```

```
        ArrayList<Integer> timeDifferences = new ArrayList<>();
```

```
        for (int clock : systemClocks) {
```

```
            timeDifferences.add(averageClock - clock);
```

```
        }
```

```
        int timeAdjustment = 0;
```

```
for (int difference : timeDifferences) {  
    timeAdjustment += difference;  
}  
  
timeAdjustment /= timeDifferences.size();  
  
masterClock = averageClock - timeAdjustment;  
  
System.out.print("Updated system clocks: ");  
for (int clock : systemClocks) {  
    System.out.print((clock - timeAdjustment) + " ");  
}  
  
System.out.println();  
  
System.out.println("Master clock: " + masterClock);  
}  
}
```

Practical 5

```
import java.io.*;
```

```
import java.util.*;
```

```
class tokenring {
```

```
    public static void main(String args[]) throws Throwable {
```

```
        Scanner scan = new Scanner(System.in);
```

```
        System.out.println("Enter the num of nodes:");
```

```
        int n = scan.nextInt();
```

```
        int m = n - 1;
```

```
        int token = 0;
```

```
        int ch = 0, flag = 0;
```

```
        for (int i = 0; i < n; i++) {
```

```
            System.out.print(" " + i);
```

```
        }
```

```
        System.out.println(" " + 0);
```

```
        do {
```

```
            System.out.println("Enter sender:");
```

```
            int s = scan.nextInt();
```

```
            System.out.println("Enter receiver:");
```

```
            int r = scan.nextInt();
```

```
System.out.println("Enter Data:");
```

```
int a = scan.nextInt();
```

```
System.out.print("Token passing:");
```

```
for (int i = token, j = token; (i % n) != s; i++, j = (j + 1) % n) {
```

```
    System.out.print(" " + j + "->");
```

```
}
```

```
System.out.println(" " + s);
```

```
System.out.println("Sender " + s + " sending data: " + a);
```

```
for (int i = s + 1; i != r; i = (i + 1) % n) {
```

```
    System.out.println("data " + a + " forwarded by " + i);
```

```
}
```

```
System.out.println("Receiver " + r + " received data: " + a + "\n");
```

```
token = s;
```

```
do {
```

```
    try {
```

```
        if (flag == 1)
```

```
            System.out.print("Invalid Input!!...");
```

```
System.out.print("Do you want to send again?? enter 1 for Yes and 0 for No : ");
```

```
ch = scan.nextInt();
```

```
        if (ch != 1 && ch != 0)

            flag = 1;

        else

            flag = 0;

    } catch (InputMismatchException e) {

        System.out.println("Invalid Input");

    }

} while (ch != 1 && ch != 0);

} while (ch == 1);

}

}
```

Practical 6

FILE 1 — Bully.java

```
import java.util.Scanner;

public class Bully {

    static boolean[] state = new boolean[5];

    public static void up(int up) {

        if (state[up - 1]) {
            System.out.println("process " + up + " is already up");
        } else {

            state[up - 1] = true;
            System.out.println("process " + up + " held election");

            for (int i = up; i < 5; i++) {
                System.out.println("election message sent from process " + up + " to process " + (i + 1));
            }

            for (int i = up + 1; i <= 5; i++) {
                if (state[i - 1]) {
                    System.out.println("alive message sent from process " + i + " to process " + up);
                    break;
                }
            }
        }
    }
}
```

```

public static void down(int down) {

    if (!state[down - 1]) {
        System.out.println("process " + down + " is already down");
    } else {
        state[down - 1] = false;
    }
}

public static void mess(int mess) {

    if (state[mess - 1]) {

        if (state[4]) {
            System.out.println("OK");
        } else {

            System.out.println("process " + mess + " election");

            for (int i = mess; i < 5; i++) {
                System.out.println("election send from process " + mess + " to process " + (i + 1));
            }

            for (int i = 5; i >= mess; i--) {
                if (state[i - 1]) {
                    System.out.println("coordinator message sent from process " + i + " to all");
                    break;
                }
            }
        }
    }
}

```



```
    } else {  
        System.out.println("process " + mess + " is down");  
    }  
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    for (int i = 0; i < 5; i++)
```

```
        state[i] = true;
```

```
    int choice;
```

```
    do {
```

```
        System.out.println("\n1 Up process\n2 Down process\n3 Send message\n4 Exit");
```

```
        choice = sc.nextInt();
```

```
        switch (choice) {
```

```
            case 1:
```

```
                System.out.println("Enter process to bring up:");
```

```
                int up = sc.nextInt();
```

```
                up(up);
```

```
                break;
```

```
            case 2:
```

```
                System.out.println("Enter process to bring down:");
```

```
                int down = sc.nextInt();
```

```
                down(down);
```

```

        break;

    case 3:
        System.out.println("Enter sender process:");
        int mess = sc.nextInt();
        mess(mess);
        break;
    }

    } while (choice != 4);
}

```

FILE 2 — Ring.java

```

import java.util.Scanner;

class Rr {
    public int index;
    public int id;
    public int f;
    String state;
}

public class Ring {

    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);

        Rr proc[] = new Rr[10];
    }
}

```

```

for (int i = 0; i < proc.length; i++)
    proc[i] = new Rr();

System.out.println("Enter number of processes:");
int num = in.nextInt();

for (int i = 0; i < num; i++) {
    proc[i].index = i;
    System.out.println("Enter process id:");
    proc[i].id = in.nextInt();
    proc[i].state = "active";
}

proc[num - 1].state = "inactive";
System.out.println("Coordinator: " + proc[num - 1].id);

while (true) {

    System.out.println("\n1 Election\n2 Quit");
    int ch = in.nextInt();

    for (int i = 0; i < num; i++)
        proc[i].f = 0;

    if (ch == 1) {

        System.out.println("Enter initiator process:");
        int init = in.nextInt();

        int start = init;
        int temp = init + 1;

```

```

int arr[] = new int[10];

int k = 0;

while (start != temp) {

    if ("active".equals(proc[temp].state) && proc[temp].f == 0) {

        System.out.println("Process " + proc[start].id + " sends to " + proc[temp].id);

        proc[temp].f = 1;
        start = temp;
        arr[k++] = proc[temp].id;
    }

    temp++;

    if (temp == num)
        temp = 0;
}

int max = -1;

for (int i = 0; i < k; i++)
    if (arr[i] > max)
        max = arr[i];

System.out.println("New coordinator is: " + max);
}

else {

```

```
        break;
    }
}
}
```